

Pipeline DataOps para Predicción de Churn en Telecomunicaciones

Evaluación Parcial N°2 · ITY1101 Gestión de Datos para IA · Sección 003V

Benjamín Heresmann — Ingeniero de Datos

ingesta · validación · arquitectura

Diego Hernández — Ingeniero de BD / DataOps

limpieza · carga · seguridad · deploy

Caso: Telco Customer Churn · 2 de junio de 2026

Qué veremos

1. El problema y la necesidad del proyecto
2. DataOps vs. enfoque tradicional
3. Arquitectura cloud desacoplada
4. Ciclo de vida del dato (pipeline)
5. Metodología y planificación
6. Las 4 etapas del pipeline
7. Plan de seguridad y KPIs
8. Demo funcional en vivo
9. Conclusiones y próximos pasos
10. Preguntas

15 min presentación

demo en vivo

5 min preguntas

El problema: la fuga de clientes

- Una telco pierde clientes (churn) cada mes.
- Captar un cliente nuevo cuesta mucho más que retener uno.
- Para retener hay que anticipar quién se va a ir.
- Anticipar exige datos limpios, confiables y trazables → ahí entra el pipeline.

26,5%
TASA DE CHURN EN EL DATASET

1.869 de 7.043 clientes abandonaron.
Cada uno = ingresos recurrentes perdidos.

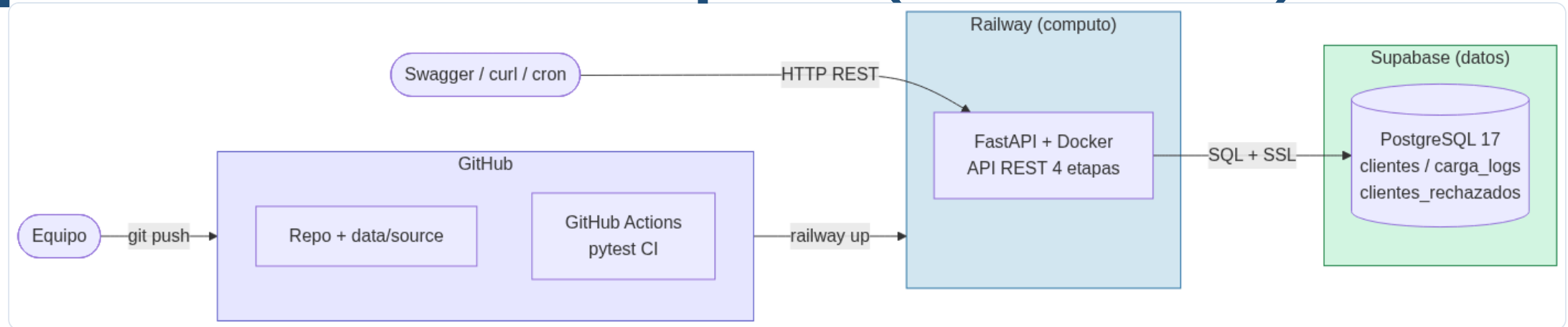
Sin datos de calidad no hay modelo de IA que funcione. El pipeline es el cimiento.

¿Por qué DataOps y no el enfoque tradicional?

Dimensión	Enfoque tradicional	DataOps (nuestro proyecto)
Procesamiento	Excel manual, copy-paste	Scripts automatizados (4 etapas)
Errores	Invisibles, se detectan tarde	Validación explícita + registro de rechazos
Reproducibilidad	"Funciona en mi PC"	Docker + idempotente (mismo input → mismo estado)
Trazabilidad	Sin historial	Cada ejecución auditada en BD (carga_logs)
Colaboración	Un archivo, una persona	Git + GitHub + CI automático
Despliegue	Local, frágil	Cloud (Railway + Supabase), API pública

DataOps = construir un sistema reproducible, observable y colaborativo, no solo "correr un script".

Arquitectura cloud desacoplada (no monolito)



GitHub

repo + CI (pytest en cada push)

Railway

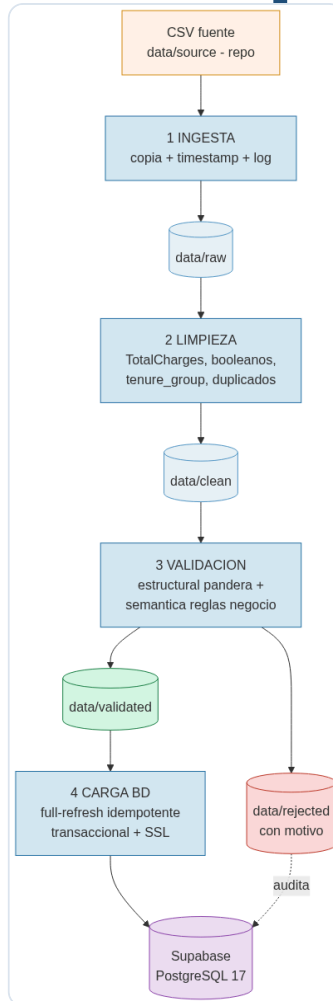
cómputo: FastAPI + Docker

Supabase

datos: PostgreSQL 17 + SSL

Tres servicios independientes. Si Railway cae, la BD sigue; se escala la API sin tocar los datos. Cumple lo pedido: no monolítico, desplegado en Railway, con Supabase.

El pipeline: 4 etapas desacopladas



Cada etapa es un endpoint REST independiente:

- `POST /pipeline/ingest`
- `POST /pipeline/clean`
- `POST /pipeline/validate`
- `POST /pipeline/load`
- `POST /pipeline/run` → orquesta las 4

Si una etapa falla, las anteriores ya dejaron su salida persistida → se retoma desde ahí.

PMBOK híbrido (predictivo + adaptativo)

Predictivo

Requisitos fijos, dependencias claras

- Modelo de BD y DDL
- Esquema de validación
- Dockerfile e integración

→ cronograma, hitos, Carta Gantt

Adaptativo

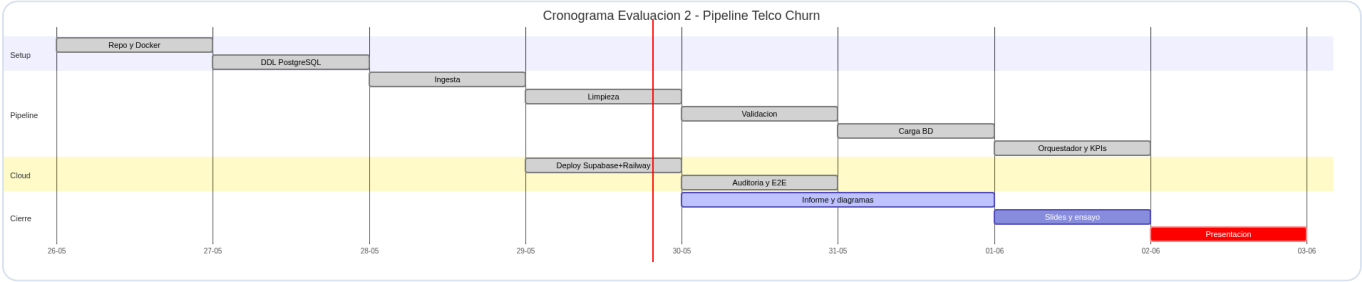
Exploratorio, iterativo

- Reglas de validación semántica
- Afinamiento de KPIs
- Ensayo de la demo

→ ciclos cortos, revisión cruzada

Cascada pura sería rígida; ágil pura, caótica para las dependencias técnicas. El híbrido planifica lo estable e itera lo incierto. Seguimiento con Trello.

WBS y Carta Gantt



26–28 may	Setup + DDL + pipeline (4 etapas)
29 may	Deploy cloud (Supabase + Railway)
29 may	Auditoría + tests E2E
30 may–1 jun	Informe, diagramas, slides
2 jun	Entrega + defensa

Hitos: H1 stack operativo · H2 pipeline end-to-end · H3 documentación y demo.

1 Ingesta

Qué hace: captura el CSV fuente y lo deposita en `data/raw/` con sello temporal, sin transformarlo.

Herramienta: Python (`shutil` + `pandas`), logging con conteo de filas.

Automatización clave: un comando deja el dato listo y trazado para la siguiente etapa.

Decisión técnica

Ingesta por lotes (batch), no streaming: el dato es un CSV estático →

Kafka sería sobreingeniería.

Dataset versionado en el repo → reproducible y sin dependencias externas en la demo.

2 Limpieza y transformación

- Bug clásico resuelto: `TotalCharges` venía como texto con vacíos → a numérico, imputando 0 a clientes con `tenure=0`.
- Yes/No → booleano.
- Feature derivada `tenure_group` (5 rangos).
- Eliminación de duplicados por `customerID`.

Principio

Se trabaja sobre copia, nunca sobre el crudo. La limpieza normaliza; no decide qué es válido (eso es la etapa 3).

3 Validación estructural y semántica

Estructural — `pandera`

- Tipos por columna
- Rangos (`tenure` 0–100)
- Valores permitidos
- Regex de `customerID`

Semántica — reglas de negocio

- Si `InternetService=No` → `servicios` = "No internet service"
- `PhoneService` ↔ `MultipleLines`
- Coherencia `TotalCharges`

Válidos → `data/validated/`. Inválidos → `data/rejected/` con su motivo, auditados en la tabla `clientes_rechazados`.

4 Carga a base de datos

- Persiste en Supabase PostgreSQL (SQLAlchemy + SSL).
- Full-refresh idempotente: `TRUNCATE` + insert en una transacción → reejecutar da siempre el mismo estado.
- Atómico: ante fallo, `ROLLBACK`.
- Auditoría en `carga_logs` (nunca se trunca).

Manejo de anomalías

Conexión caída → estado "ERROR" en log · violación de constraint → rollback · inválidos → separados y auditados, nunca descartados en silencio.

Seguridad: legal + técnica

Normas legales

- Ley 19.628 — datos personales (Chile)
- Ley 21.459 — delitos informáticos
- Tratamiento limitado al propósito

Técnicas

- Cifrado en tránsito (SSL a la BD)
- Control de acceso (rol solo-SELECT)
- Secretos en variables de entorno
- Anti-inyección (SQL parametrizado)
- Enmascaramiento / logs sin PII

enmascaramiento

control de acceso

cifrado

auditoría

KPIs del pipeline

KPI	Umbral de alerta
Latencia total	> 30 s
Tasa de validez	< 95%
Compleitud por columna	< 95%
Volumen procesado	< 5.000
Estado de ejecución	≠ OK

Consultables en vivo

Persistidos en `carga_logs` y expuestos por la API:

`GET /kpis/resumen`

`GET /kpis/last`

En producción se integran a Grafana / alertas.

Demo: pipeline en producción

Ejecutamos `POST /pipeline/run` en vivo → carga 7.043 clientes en Supabase · luego `GET /kpis/resumen` y `GET /rechazados` .

URL: telco-api-production-e466.up.railway.app/docs

Conclusiones y próximos pasos

Lo logrado

- Pipeline DataOps de 4 etapas, reproducible y trazable.
- Desplegado en la nube, no monolítico.
- Probado end-to-end + CI verde.

Próximos pasos

- Evaluación 3: entrenar modelo de clasificación sobre churn .
- Variables clave: tenure , Contract , MonthlyCharges .
- Orquestación (Airflow) y dashboard de KPIs.

La tabla `clientes` en Supabase ya queda lista como fuente de entrenamiento para el modelo de IA.

¡Gracias!

¿Preguntas?

Repositorio: github.com/BenjaminHeresmann/telco-churn-pipeline

API: telco-api-production-e466.up.railway.app/docs

Benjamín Heresmann · Diego Hernández — Sección 003V